

Python collections Module

Comprehensive Study Material

Machine Learning Bootcamp | Instructor: Wahid Vincenzo

LEARNING OBJECTIVES

- * Master all major components of Python's collections module
- * Understand time/space complexity trade-offs
- * Apply collections to real Machine Learning workflows
- * Build intuition for choosing the right data structure
- * Prepare for technical interviews and coding challenges

7

Collection
Types Covered

30+

Code
Examples

5

Practical
Exercises

3

ML
Applications

Table of Contents

1	Introduction to the collections Module	3
2	Counter — Frequency Counting	4
3	defaultdict — Auto-Default Values	6
4	OrderedDict — Order-Aware Dictionary	8
5	deque — Double-Ended Queue	10
6	namedtuple — Lightweight Records	12
7	ChainMap — Layered Lookups	14
8	UserDict / UserList / UserString	16
9	Comparison & Decision Guide	17
10	ML Applications Summary	18
11	Exam Preparation Checklist	19

1 Introduction to the collections Module

Python's **collections** module is part of the standard library and provides specialized container datatypes as alternatives to Python's general-purpose built-ins such as **dict**, **list**, **set**, and **tuple**. These alternatives are optimized for specific use cases, offering better performance, richer interfaces, and cleaner code.

Why Use the collections Module?

- Provides **specialized** data structures for common programming patterns
- All implementations are **C-optimized** under the hood — faster than hand-rolled Python alternatives
- Reduces boilerplate and makes code more **readable and maintainable**
- Each type is a **subclass** of a built-in type — interoperable with existing code
- Critical for **Machine Learning** workflows: feature engineering, NLP, dataset management

Module Overview

Collection Type	Base Type	Key Advantage	Primary Use Case
Counter	dict	Auto-counting	Frequency analysis
defaultdict	dict	No KeyError	Grouping / accumulating
OrderedDict	dict	move_to_end()	LRU cache, ordered ops
deque	object	O(1) both ends	Queues, sliding windows
namedtuple	tuple	Named fields	Readable records
ChainMap	MutableMapping	Layered lookup	Config management
UserDict/List	UserDict	Safe subclassing	Custom collections

How to Import

```
from collections import Counter, defaultdict, OrderedDict
from collections import deque, namedtuple, ChainMap
from collections import UserDict, UserList, UserString

# Or import everything
import collections
```

2 Counter — Frequency Counting

Counter is a dict subclass designed for counting hashable objects. Elements are stored as dictionary keys and their counts as values. Missing keys return 0 rather than raising a `KeyError`.

Definition

```
from collections import Counter

# Multiple ways to create a Counter
c1 = Counter('banana')           # From string
c2 = Counter(['a', 'b', 'a', 'c', 'b']) # From list
c3 = Counter({'a': 3, 'b': 2})     # From dict
c4 = Counter(a=3, b=2, c=1)       # From kwargs

print(c1) # Counter({'a': 3, 'n': 2, 'b': 1})
```

Key Methods

Method	Description	Example
<code>most_common(n)</code>	Returns n most common elements as (element, count) pairs	<code>c.most_common(3)</code>
<code>elements()</code>	Iterator over elements repeating each by its count	<code>list(c.elements())</code>
<code>subtract(iterable)</code>	Subtract counts (allows negative counts)	<code>c.subtract(['a', 'a'])</code>
<code>update(iterable)</code>	Add counts from another iterable or mapping	<code>c.update('aab')</code>
<code>total()</code>	Sum of all counts (Python 3.10+)	<code>c.total()</code>
<code>c[missing_key]</code>	Returns 0, not <code>KeyError</code>	<code>c['z'] == 0</code>

Arithmetic Operations

```
c1 = Counter(a=5, b=3, c=2)
c2 = Counter(a=2, b=4, d=1)

c1 + c2 # Addition: Counter({'a':7, 'b':7, 'c':2, 'd':1})
c1 - c2 # Subtraction: Counter({'a':3, 'c':2}) [negatives dropped]
c1 & c2 # Intersection (min): Counter({'a':2, 'b':3})
c1 | c2 # Union (max): Counter({'a':5, 'b':4, 'c':2, 'd':1})
```

ML Use Case: Word Frequency (NLP)

Counter is ideal for bag-of-words models, building vocabulary indices, computing term frequencies (TF), and analyzing class imbalance in datasets.

Advantages & Limitations

Advantages	Limitations
------------	-------------

One-line frequency counting
Arithmetic operations on counts
Missing keys return 0 (no `KeyError`)
Memory efficient — only stores non-zero counts

Not suitable for floating-point counts
No thread-safety guarantees
`most_common()` is $O(n \log k)$, not $O(1)$
Cannot store unhashable items as keys

3 defaultdict — Auto-Default Values

defaultdict is a dict subclass that calls a factory function to supply missing values. When a key is accessed that doesn't exist, instead of raising a `KeyError`, it calls `default_factory()` to create the default value.

Definition & Syntax

```
from collections import defaultdict

dd = defaultdict(default_factory)

# Common factory functions:
defaultdict(int)          # default = 0
defaultdict(list)         # default = []
defaultdict(set)          # default = set()
defaultdict(str)          # default = ''
defaultdict(lambda: 'N/A') # custom default
```

Factory Functions & Their Defaults

default_factory	Default Value	Best For
int	0	Counting, frequency tables
float	0.0	Accumulating float sums
list	[]	Grouping, multi-values per key
set	set()	Unique value grouping
dict	{}	Nested dictionaries
str	''	String accumulation
lambda: None	None	Optional values
lambda: [0]*n	[0, 0, ...]	Fixed-size vector per key

Classic Patterns

```
# PATTERN 1: Word count (alternative to Counter)
word_count = defaultdict(int)
for word in text.split():
    word_count[word] += 1

# PATTERN 2: Grouping by key
by_dept = defaultdict(list)
for name, dept in employees:
    by_dept[dept].append(name)

# PATTERN 3: Graph as adjacency list
graph = defaultdict(list)
graph['A'].extend(['B', 'C'])
graph['B'].append('D')
```

Important Behavior: default_factory is called without arguments

`dd.default_factory` must be callable with no args. Once a missing key is accessed, the key IS added to the dict with the default value.
Use `dict(dd)` to convert to a plain dict to prevent accidental key creation.

Inverted Index — ML Application

```
# Search engine / NLP: map each word to its document IDs
inverted_index = defaultdict(list)
for doc_id, text in documents.items():
    for word in text.split():
        inverted_index[word].append(doc_id)

# Query: find docs containing ALL query words
results = set.intersection(*[set(inverted_index[w]) for w in query])
```

4 OrderedDict — Order-Aware Dictionary

OrderedDict is a dict subclass that maintains insertion order and provides order-aware operations not available in regular dicts. While Python 3.7+ dicts also maintain insertion order, OrderedDict provides unique operations like **move_to_end()** and order-sensitive equality comparison.

Key Differences from Regular dict

Feature	dict (Python 3.7+)	OrderedDict
Insertion order preserved	Yes	Yes
move_to_end()	No	Yes
popitem(last=False)	No	Yes
Order-sensitive equality	No	Yes
Memory overhead	Lower	Slightly higher
Best for	General use	LRU cache, ordered ops

move_to_end() — The Killer Feature

```
od = OrderedDict(a=1, b=2, c=3, d=4)
print(list(od.keys()))      # ['a', 'b', 'c', 'd']

od.move_to_end('b')         # Move 'b' to the END
print(list(od.keys()))      # ['a', 'c', 'd', 'b']

od.move_to_end('d', last=False) # Move 'd' to the FRONT
print(list(od.keys()))      # ['d', 'a', 'c', 'b']

# Order-sensitive equality
od1 = OrderedDict([("a", 1), ("b", 2)])
od2 = OrderedDict([("b", 2), ("a", 1)])
print(od1 == od2)          # False — order matters!
```

LRU Cache — Classic OrderedDict Pattern


```
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key) # Mark as recently used
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False) # Evict LRU
```

Real-World Applications

- * LRU/MRU cache implementation ($O(1)$ get/put)
- * Maintaining processing order in pipelines
- * Building history/undo stacks
- * Implementing round-robin schedulers

5 deque — Double-Ended Queue

deque (pronounced *deck*) is a thread-safe, memory-efficient double-ended queue that supports appending and popping from *both ends* in $O(1)$ time. It is implemented as a doubly-linked list of fixed-size blocks.

Time Complexity Comparison

Operation	list	deque	Why deque wins
append (right)	$O(1)$	$O(1)$	Both equally fast
appendleft (left)	$O(n)$	$O(1)$	list must shift all elements
pop (right)	$O(1)$	$O(1)$	Both equally fast
popleft (left)	$O(n)$	$O(1)$	list.pop(0) shifts all elements
Random access [i]	$O(1)$	$O(n)$	list is array-backed
rotate(k)	$O(n)$	$O(k)$	deque has built-in rotation
len()	$O(1)$	$O(1)$	Both $O(1)$

Key Methods & Parameters

```
from collections import deque

dq = deque([1, 2, 3], maxlen=5) # maxlen creates a circular buffer

dq.append(4)           # Add to right  → deque([1, 2, 3, 4])
dq.appendleft(0)       # Add to left   → deque([0, 1, 2, 3, 4])
dq.extend([5,6])       # Extend right  → deque([2, 3, 4, 5, 6]) [0,1 evicted!]
dq.pop()               # Remove right  → 6
dq.popleft()           # Remove left   → 2
dq.rotate(2)           # Rotate right  → elements shift right by 2
dq.rotate(-1)          # Rotate left   → elements shift left by 1
dq.reverse()           # Reverse in place
dq.count(3)            # Count occurrences of 3
```

maxlen — Sliding Window / Ring Buffer

```
# Sliding window average — common in time-series ML
window = deque(maxlen=3)
data = [10, 20, 30, 40, 50]

for val in data:
    window.append(val) # Oldest auto-evicted when full
    print(f'Window: {list(window)}, Avg: {sum(window)/len(window):.1f}')
```

Output:

```
# Window: [10], Avg: 10.0
# Window: [10, 20], Avg: 15.0
# Window: [10, 20, 30], Avg: 20.0
# Window: [20, 30, 40], Avg: 30.0 ← 10 evicted
# Window: [30, 40, 50], Avg: 40.0 ← 20 evicted
```

ML Applications of deque

- * BFS/DFS graph traversal (neural net topology search)
- * Replay memory buffer in Reinforcement Learning (DQN)
- * Sliding window feature engineering for time-series
- * Rate limiting / sliding window counters
- * Undo/redo history with fixed capacity

6 namedtuple — Lightweight Readable Records

namedtuple creates tuple subclasses with named fields. It provides the immutability and memory efficiency of tuples combined with the readability of named attribute access — without the overhead of a class.

Syntax

```
from collections import namedtuple

# Method 1: List of field names
Point = namedtuple('Point', ['x', 'y'])

# Method 2: Space-separated string
Color = namedtuple('Color', 'red green blue')

# Method 3: With defaults (Python 3.6.1+)
Config = namedtuple('Config', ['lr', 'epochs', 'batch'],
                    defaults=[0.001, 100, 32])
```

namedtuple vs Alternatives

Feature	tuple	namedtuple	dict	dataclass
Immutable	Yes	Yes	No	Optional
Named access	No	Yes	Yes	Yes
Memory efficient	Yes	Yes	No	No
Iteration	Yes	Yes	Yes	No
Methods	No	Built-ins	No	Yes
Python version	All	2.6+	All	3.7+

Built-in Methods

```
Employee = namedtuple('Employee', ['name', 'dept', 'salary'])
emp = Employee('Alice', 'Engineering', 95000)

# Access methods
emp.name          # 'Alice' (named access)
emp[0]            # 'Alice' (index access)
name, dept, sal = emp      # unpacking

# Utility methods
emp._asdict()      # OrderedDict of fields
emp._replace(salary=110000) # New copy with modification
Employee._fields   # ('name', 'dept', 'salary')
Employee._make(['Bob', 'HR', 70000]) # Create from iterable
```

Best Practice: Extend namedtuple with custom methods

```
class BoundingBox(namedtuple('BoundingBox', ['x1','y1','x2','y2'])):
    def area(self):
        return (self.x2 - self.x1) * (self.y2 - self.y1)
    def iou(self, other):
        ... # Intersection over Union calculation
```

7 ChainMap — Layered Dictionary Lookups

ChainMap groups multiple dictionaries into a single logical view. Lookups search through the maps *in order* until the key is found. Writes always go to the *first* map only, making it ideal for layered configuration and variable scoping.

Lookup Mechanism

```
from collections import ChainMap

# Priority: local_env > user_settings > global_defaults
defaults = {'debug': False, 'timeout': 30, 'log': 'WARNING'}
user      = {'timeout': 60, 'log': 'INFO'}
local     = {'debug': True}

config = ChainMap(local, user, defaults)

config['debug']      # True    (from local)
config['timeout']    # 60     (from user; not in local)
config['log']        # 'INFO' (from user; not in local)
config['x']          # KeyError: not in any map
```

Key Methods

Method / Property	Description
cm.maps	List of underlying dicts in priority order
cm.parents	ChainMap of all maps except the first
cm.new_child(m)	New ChainMap with m as highest-priority map
cm[key] = value	Writes only to the first (highest-priority) map
del cm[key]	Deletes only from the first map
len(cm)	Number of unique keys across all maps
list(cm)	All unique keys

Variable Scoping with ChainMap

```
# Python's own variable lookup works exactly like ChainMap!
# local scope → enclosing → global → built-in

scope = ChainMap()          # empty global scope
scope['x'] = 10

# Enter function scope
scope = scope.new_child({'x': 99, 'y': 5})
print(scope['x']) # 99 (local shadows global)
print(scope['y']) # 5

# Exit function scope
scope = scope.parents
print(scope['x']) # 10 (global unchanged)
```

ChainMap vs dict.update()

dict.update() merges by copying — changes to original dicts are NOT reflected.

ChainMap uses the original dicts as views — changes ARE reflected dynamically.

ChainMap is also O(1) to create (no copying), while update() is O(n).

8 UserDict / UserList / UserString

These three classes provide **safe base classes** for creating custom collection types. When you subclass built-in **dict**, **list**, or **str** directly, Python's C implementation may bypass your overridden methods. These wrappers avoid that pitfall by keeping data in a **.data** attribute.

The Subclassing Problem

```
# DANGER: Subclassing dict directly – methods may be bypassed!
class MyDict(dict):
    def __setitem__(self, key, value):
        print(f'Setting {key}')
        super().__setitem__(key, value)

d = MyDict(a=1) # __setitem__ NOT called! Python bypasses it.

# SAFE: Subclassing UserDict – always goes through your code
class MyDict(UserDict):
    def __setitem__(self, key, value):
        print(f'Setting {key}') # Always called
        super().__setitem__(key, value)

d = MyDict(a=1) # Prints 'Setting a' ■
```

UserDict — Custom Validated Dictionary

```
class ReadOnlyDict(UserDict):
    """A dictionary that doesn't allow modifications."""
    def __setitem__(self, key, value):
        raise TypeError('ReadOnlyDict does not support item assignment')
    def __delitem__(self, key):
        raise TypeError('ReadOnlyDict does not support deletion')

rod = ReadOnlyDict({'model': 'ResNet50', 'version': '1.0'})
print(rod['model']) # 'ResNet50'
rod['model'] = 'VGG' # TypeError!
```

UserList — Typed List

```
class TypedList(UserList):
    """A list that only accepts a specific type."""
    def __init__(self, dtype, initlist=None):
        self.dtype = dtype
        super().__init__(initlist or [])

    def append(self, item):
        if not isinstance(item, self.dtype):
            raise TypeError(f'Expected {self.dtype.__name__}')
        super().append(item)

int_list = TypedList(int)
int_list.append(42) # OK
int_list.append('oops') # TypeError!
```


9 Comparison & Decision Guide

Quick Decision Flowchart

Use these guiding questions to choose the right collection:

Do you need to COUNT elements?	→ Counter
Do you need a dict that GROUPS values?	→ defaultdict(list)
Do you need a dict without KeyError?	→ defaultdict(int/...)
Do you need ORDER-SENSITIVE dict ops?	→ OrderedDict
Do you need FAST queue operations?	→ deque
Do you need a SLIDING WINDOW buffer?	→ deque(maxlen=n)
Do you need READABLE tuple records?	→ namedtuple
Do you need LAYERED config/scoping?	→ ChainMap
Do you need CUSTOM dict/list behavior?	→ UserDict / UserList

Complete Feature Comparison Matrix

Feature	Counter	defaultdict	OrderedDict	deque	namedtuple	ChainMap
Base type	dict	dict	dict	—	tuple	MutableMap
O(1) insert	Yes	Yes	Yes	Both ends	Yes (append)	Yes
O(1) delete	Yes	Yes	Yes	Both ends	No (immutable)	First map only
Key-based access	Yes	Yes	Yes	No	No (index)	Yes
Ordered	3.7+	3.7+	Yes	Yes	Yes	Yes
Mutable	Yes	Yes	Yes	Yes	No	First map
Thread-safe	No	No	No	Yes	Yes	No
Memory	Medium	Medium	Medium+	Low	Very Low	Low

10 ML Applications Summary

Python's collections module appears throughout the Machine Learning stack — from data preprocessing to model management. Here are the most common applications:

Counter

- Bag-of-words text representation
- Class imbalance analysis
- Vocabulary building for NLP models
- Top-k feature selection

defaultdict

- Building inverted indices for search
- Grouping samples by class label
- Graph adjacency lists for GNNs
- Accumulating gradients per parameter

OrderedDict

- LRU model/embedding cache
- Maintaining pipeline step order
- Ordered hyperparameter tracking

deque

- Replay memory buffer (DQN)
- Sliding window feature engineering
- BFS graph traversal
- Rolling loss/metric tracking

namedtuple

- Dataset record representation
- Bounding box definitions (object detection)
- Model configuration containers
- Experiment result tracking

ChainMap

- Layered hyperparameter resolution
- Environment-specific model configs
- Variable scoping in interpreters

Code Pattern: Complete ML Pipeline Snippet

```
from collections import Counter, defaultdict, deque, namedtuple, ChainMap

# 1. Load & structure data
Sample = namedtuple('Sample', ['features', 'label', 'weight'])
dataset = [Sample([1.0, 2.0], 'cat', 1.0), ...]

# 2. Analyze class distribution
label_dist = Counter(s.label for s in dataset)

# 3. Group by label
by_label = defaultdict(list)
for s in dataset:
    by_label[s.label].append(s)

# 4. Replay buffer for RL
replay = deque(maxlen=10000)

# 5. Layered config
hp = ChainMap(exp_overrides, model_defaults, global_defaults)
```

11 Exam Preparation Checklist

Key Concepts to Master

Counter

- Creating from strings, lists, dicts, kwargs
- `most_common(n)` — what does it return and in what order?
- Arithmetic: `+`, `-`, `&`, `|` and how they handle negatives
- Missing key behavior: returns 0 vs `KeyError`
- `elements()` iterator and what it produces

defaultdict

- How `default_factory` works — called with no args
- Difference from `setdefault()` and `get(key, default)`
- Writing grouping/accumulation code from scratch
- Why accessing a missing key ADDS it to the dict

OrderedDict

- `move_to_end(key, last=True/False)` behavior
- Order-sensitive vs order-insensitive equality
- `popitem(last=True)` vs `popitem(last=False)`
- LRU cache implementation step-by-step

deque

- $O(1)$ for `append/appendleft/pop/popleft`
- `maxlen` behavior — what gets evicted and when
- `rotate(n)` direction for positive vs negative `n`
- Why deque beats list for queue operations

namedtuple

- Three ways to define field names
- `_asdict()`, `_replace()`, `_make()`, `_fields`
- `defaults` parameter — applied from right
- Extending namedtuple with custom methods

ChainMap

- Lookup order and which map writes go to
- `new_child()` vs creating a new ChainMap manually
- `.maps` and `.parents` attributes
- ChainMap vs `dict.update()` — key difference

Common Interview Questions

1. Implement an LRU cache using OrderedDict — what is the time complexity?
2. What is the difference between Counter and defaultdict(int)?
3. When would you use a deque instead of a list? Give complexity reasons.
4. How does ChainMap differ from merging dicts with update()?
5. Why use namedtuple instead of a regular class or dict?
6. How can you safely extend a dict's behavior without subclassing dict?
7. Implement a sliding window maximum using deque in $O(n)$ time.
8. What happens when you access a missing key in defaultdict vs Counter?

SUMMARY MANTRA

Count things → Counter
Group things → defaultdict(list)
Order matters → OrderedDict
Queue/Sliding → deque(maxlen=n)
Readable records → namedtuple
Layer configs → ChainMap
Custom containers → UserDict / UserList